

Strings and Character Array Manipulation in C

> **Definition to Remember**

> In C
library

1. Concept Overview & Fundamentals

Strings and Character Array Manipulation in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

- Co

2. Technical Architecture & Memory Mechanics

Memory Layout & Storage Organization

```text

+-----

### Core Architecture Breakdown

| Component / Concept | Technical Specification | Operational Characteristics | Performance / Complexity |

| :--- | :

---

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
```c
#include <stdio.h>

// Core standard implementation pattern
int main() {

// Core logic execution
printf("Program execution started.\n");

return status;
}
```

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.

- **E**

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: break keyword loop

Problem Description & Objective:

****C Code Implementation:****

```
#include <stdio.h>
```

```
int main() {
```

```
while(1) {
```

```
printf("Enter text (type 'exit' to quit): ");
```

```
// Check exit condition
```

```
printf("You entered: %s\n", text);
```

```
printf("Program Ended.\n");
```

```
return 0;
```

```
---
```

Problem 2: date format

****Problem Description & Objective:****

****C Code Implementation:****

```
#include <stdio.h>
```

```
int main() {
```

```
/*
```

```
int day = 1;
```

```
int mon
```

```
printf("Formatted Date: %02d/%02d/%d\n",
```

```
day, m
```

```
return 0;
```

```
}
```

```
---
```

Problem 3: fgets puts

****Problem Description & Objective:****

Implem

****C Code Implementation:****

```
```c
```

```
#include <stdio.h>
```

```
int main() {
```

```
/*
```

```
char text[100];
```

```
printf("Enter a line of text:\n");
```

```
fgets(text, sizeof(text), stdin);
```

```
printf("\nYou entered:\n");
```

```
puts(text);
```

```
return 0;
```

```
}
```

```

```

#### Problem 4: login system

**\*\*Problem Description & Objective:\*\***

Implemen

**\*\*C Code Implementation:\*\***

```
```c
```

```
#include <stdio.h>
```

```
#includ
```

```
int main() {
```

```
/*
```

```
printf("Enter password: ");
```

```
scanf("
```

```
if(strcmp(storedPassword, userPassword) == 0) {
```

```
printf("
```

```
return 0;
```

```
}
```

```
---
```

Problem 5: password checker

****Problem Description & Objective:****

Implemen

****C Code Implementation:****

```
```c
```

```
#include <stdio.h>
```

```
int main() {
```

```
do {
```

```
printf("Enter password: ");
```

```
if(strcmp(password, "admin123") != 0) {
```

```
} while(strcmp(password, "admin123") != 0);
```

```
printf("Valid Password Entered.\n");
```

```
return 0;
```

```

```

### Problem 6: reverse string function

**\*\*Problem Description & Objective:\*\***

**\*\*C Code Implementation:\*\***

```
#include <stdio.h>
```

```
// Function Definition
```

```
int start = 0;
```

```
while(start < end) {
```

```
temp = str[start];
```

```
start++;
```

```
int main() {
```

```
printf("Enter a string: ");
```

```
reverseString(str);
```

```
printf("Reversed String: %s\n", str);
```

```
return 0;
```

```

```

### **Problem 7: string palindrome**

**\*\*Problem Description & Objective:\*\***

**\*\*C Code Implementation:\*\***

```
#include <stdio.h>
```

```
int main() {
```

```
char str[100];
```

```
printf("Enter a string: ");
```

```
// Remove newline character
```

```
end = strlen(str) - 1;
```

```
while(start < end) {
```

```
if(str[start] != str[end]) {
```

```
start++;
```

```
if(isPalindrome == 1) {
```

```
return 0;
```

```

```

### **Problem 8: tic tac toe board**

**\*\*Problem Description & Objective:\*\***

**\*\*C Code Implementation:\*\***

```
#include <stdio.h>
```

```
int main() {
```

```
char board[3][3] = {
 {'X', 'O', 'X'},
 {'O', 'X', 'O'},
 {'X', 'O', 'X'}

};

int i, j;

printf("Tic-Tac-Toe Board:\n\n");

for(i = 0; i < 3; i++) {
 for(j = 0; j < 3; j++) {

 if(j < 2) {
 printf("%c", board[i][j]);
 }
 else {
 printf("%c\n", board[i][j]);
 }
 }
}

return 0;

```

### Problem 9: trim string

**\*\*Problem Description & Objective:\*\***

**\*\*C Code Implementation:\*\***

```
#include <stdio.h>
```

```
// Function Definition
```

```
void trim
```

```
int start = 0;
```

```
int end
```

```
// Find first non-space character
```

```
while(i
```

```
// Find last non-space character
```

```
while(e
```

```
// Shift string to beginning
```

```
for(i =
```

```
str[j] = '\0';
```

```
}
```

```
int main() {
```

```
char s
```

```
printf("Enter a string with spaces:\n");
```

```
fgets(s
```

```
trim(str);
```

```
printf("Trimmed String: '%s'\n", str);
```

```
return 0;
```

```
}
```

```

```

### **Problem 10: uppercase string**

**\*\*Problem Description & Objective:\*\***

Implem

**\*\*C Code Implementation:\*\***

```
#include <stdio.h>
```

```
int main() {
```

```
char str[100];
```

```
printf("Enter a string: ");
```

```
while(str[i] != '\0') {
```

```
str[i] = toupper(str[i]);
```

```
printf("Uppercase String:\n%s", str);
```

```
return 0;
```

```

```

## 6. Exam Scoring Points & Memory Revision

> **\*\*Must-Write Points\*\***

> **\*\*Quick Recall\*\***